

Reviewer Pack — Minimal Symplectic Volume in Affine Geometry and Circuit Mon...

Entry 5599c41978b0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 00:58:00 UTC

Minimal Symplectic Volume in Affine Geometry and Circuit Monotone Complexity

Entry ID: 5599c41978b0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 00:58:00 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): Boolean Circuits

Statement:

For any given Boolean circuit C with n inputs, the minimal symplectic volume of its associated affine scheme φ_C is linearly correlated with its monotone complexity $w_M(C)$, such that $\min_vol(\varphi_C) = \Theta(w_M(C))$.

Rationale (proposer's reasoning):

Affine geometry offers a geometric interpretation of computational complexity through algebraic varieties. The symplectic volume measures the size of an affine scheme, which could potentially capture structural properties of Boolean circuits. If a relationship between the symplectic volume and circuit monotone complexity holds, it might provide new insights into hardness of approximation problems.

Taxonomy category: AffineGeometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5616a4b78faa7351

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the circuits, the Pearson correlation coefficient between $\min_vol(\varphi_C)$ and $w_M(C)$ exceeds 0.7 with a p-value ≤ 0.05 across all 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal symplectic volume" AND "affine geometry" AND "Boolean circuits" - "monotone complexity" IN Affine Geometry AND Boolean Circuits" - "affine scheme" AND circuit monotone complexity

Top relevant hits considered: - [http://arxiv.org/abs/0802.4323v1] Non-singular affine surfaces with self-maps - [http://arxiv.org/abs/1806.08513v2] Affine differential geometry and affine rotation surfaces: algebraic surfaces invariant under non-Euclidean affine rotation - [http://arxiv.org/abs/2106.15501v3] Ramblings on the freeness of affine hypersurfaces - [http://arxiv.org/abs/2602.19269v1] Perfect complexes on finite flat affine groupoids - [http://arxiv.org/abs/1211.1111v1] Macaulay-like marked bases - [http://arxiv.org/abs/2603.25632v1] Isometric Embeddings and Hyperkähler Geometry of the Cotangent Bundle of Complex Projective Space via the Scheme of Rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=26.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import combinations

def generate_circuit(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def affine_scheme(circuit):
    n = len(circuit)
    scheme = []
    for i in range(2**n):
        if all(circuit[j] == circuit[i ^ j] for j in range(n)):
            scheme.append(i)
    return scheme

def symplectic_volume(scheme):
    n = int(math.log2(len(scheme)))
    volume = 0
    for i in range(n):
        count = sum(1 for x in scheme if (x >> i) & 1 == 1)
        volume += count * (len(scheme) - count)
    return volume

def monotone_complexity(circuit):
```

```

n = len(circuit)
complexity = 0
for i in range(n):
    if circuit[i] == 1:
        complexity += sum(1 for j in range(i+1, n) if circuit[j] == 1 and (i & j) == i)
return complexity

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
        circuit = generate_circuit(n)
        scheme = affine_scheme(circuit)
        volume = symplectic_volume(scheme)
        complexity = monotone_complexity(circuit)
        results.append((volume, complexity))

    if not results:
        return {
            "metric_name": "Pearson correlation coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "empty_results"
        }

    n = len(results)
    mean_volume = sum(v for v, _ in results) / n
    mean_complexity = sum(c for _, c in results) / n

    covariance = sum((v - mean_volume) * (c - mean_complexity) for v, c in results) / n
    variance_volume = sum((v - mean_volume)**2 for v, _ in results) / n
    variance_complexity = sum((c - mean_complexity)**2 for _, c in results) / n

    correlation_coefficient = covariance / (math.sqrt(variance_volume) * math.sqrt(variance_complexity))

    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": n,
        "n_max": max(n for _, _ in results),
        "conjecture_holds": abs(correlation_coefficient) >= 0.7,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

```

```

if all(r["conjecture_holds"] for r in results):
    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = 1.0
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.7\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e59a8ffe.py", line 89, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e59a8ffe.py", line 50, in run_trial
    scheme = affine_scheme(circuit)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e59a8ffe.py", line 24, in affine_scheme
    if all(circuit[j] == circuit[i ^ j] for j in range(n)):
        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e59a8ffe.py", line 24, in <genexpr>
    if all(circuit[j] == circuit[i ^ j] for j in range(n)):
        ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification based on the pre-registered criteria. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17222
2	preregistration	ollama_remote	glm4:latest	0	0	17065
3	novelty	ollama_remote	glm4:latest	0	0	14388
4	novelty	ollama_remote	glm4:latest	0	0	17273
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17080
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12051
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11294
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26016
9	judge	ollama_remote	glm4:latest	0	0	8766

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 141154 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5599c41978b0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5599c41978b0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5599c41978b0.tar.gz` (if generated)